Thetawave.ai

💻 策略模式與責任鏈模式的設計差異與應用

Generated by Thetawave.ai

4.1 精準區分策略＆責任鏈 Forces.mp4

🎯 策略模式與責任鏈模式的差異理解與設計推導

> 「設計模式的關鍵不在於『怎麼套用』，而是要清楚『什麼時候套用』，透過察覺問題中的約束（forces）來決定最適合的模式。」

---

1. 為何要厘清策略（Strategy）與責任鏈（Chain of Responsibility）模式的差異？

- **準確的設計決策來自於清楚理解模式的差異**
  如果無法區分兩種模式的本質差異，設計時容易出現過度設計（overdesign）或錯誤套用模式。
- **重點在「何時套用」而非「如何套用」**
  「如何套用」就是照著模式的類別結構（form）直接重構程式碼，這部分較容易掌握。
  「何時套用」需要從物件導向分析（Object-Oriented Analysis, OOA）中發現問題的「forces」，並判斷問題的本質，進而選擇合適的模式。

---

2. 重要概念：設計模式的六大元素與推理流程

元素	說明
Context	問題發生的情境
Forces	在Context中出現的約束與衝突
Problem	因forces衝突而產生的設計困難或需求
Solution	解決Problem的方案，即設計模式的form結構
Resulting Context	套用Solution後的新Context
Name	模式名稱

推理流程：
從**Context**察覺**Forces** → 明確**Problem** → 尋找對應的**Solution (form)** → 得到新的**Resulting Context**

Thetawave.ai

3. 策略模式與責任鏈模式的類別結構（form）比較

項目	策略模式（Strategy）	責任鏈模式（Chain of Responsibility）
抽象角色	Strategy介面	Handler抽象類別
實作類別	多個Concrete Strategy實作不同策略	多個Concrete Handler串成一條鏈（有「下一個處理者」參考）
Client角色	持有一個Strategy參考，委派行為給選定的策略實作類別	持有Handler鏈的起點，並將request傳入鏈中
行為執行流程	直接呼叫策略物件執行行為	Request依序由鏈中各Handler判斷是否處理，否則傳給下一個Handler
行為變動觸發	委派給固定的策略物件，行為不會因request變化而改變	根據request的內容（參數）動態決定由哪個Handler處理行為

4. 動態行為差異解析

- **策略模式**：
  Client選擇某個固定的策略物件，行為的變動是由切換策略物件決定，且重複執行時行為一致。
- **責任鏈模式**：
  Client將帶有參數的request送入Handler鏈，鏈中每個Handler依序判斷是否處理該request，行為依request內容動態改變。

5. 行為變動性的差異（Behavioral Variation, BV）

模式	行為變動性類型	說明
策略模式	**原始型行為變動性**	同一操作中存在多種行為變種（策略），可自由替換，但不依賴輸入參數的具體內容
責任鏈模式	**輸入比對型行為變動性**	依據輸入參數的型態或內容判斷由哪個Handler處理，行為觸發需先解析輸入參數才能分派行為

6. 如何判斷何時套用策略模式與責任鏈模式？

步驟	判斷重點	對應模式

定位行為變動性	找出在某操作中是否存在多種行為變種	皆適用於行為型模式
分類行為變動性	判斷行為變動是否為「可抽換多種行為」(原始型) 或「需先解析輸入參數才能決定行為」(輸入比對型)	原始型 → 策略模式 輸入比對型 → 責任鏈模式

Thetawave.ai

句型範例

行為變動性類型	句型範例
原始型行為變動性	「在某操作中，存在多種行為之一，可抽換多種行為之一，如行為 A、行為 B、行為 C，且未來會持續擴充。」
輸入比對型行為變動性	「在某操作執行時，必須先解析輸入參數（如參數名稱），每種輸入參數對應不同處理方式（例：方式 A、方式 B、方式 C），且未來會持續擴充。」

---

7. 實例分析與套用策略

案例一：清潔人員回收垃圾

- **Context**：人丟垃圾給清潔人員回收
- **需求**：垃圾種類會持續增加，且不改動既有程式碼
- **行為變動分析**：
  - 操作：清潔垃圾
  - 輸入參數：垃圾種類
  - 不同垃圾類型對應不同處理行為
  - 未來擴充新垃圾及新處理方式
- **行為變動性**：輸入比對型行為變動性
- **結論**：適用**責任鏈模式**，因為需要依輸入垃圾類型動態決定處理者

案例二：客服訊息處理

- **Context**：客戶傳送訊息給客服，客服處理訊息
- **需求**：服務模式有多種，持續擴充且不修改既有程式碼
- **行為變動分析**：
  - 操作：傳訊息
  - 參數：訊息內容（不影響行為判斷）
  - 多種處理模式（策略）
  - 行為變動不依賴訊息類型
- **行為變動性**：原始型行為變動性
- **結論**：適用**策略模式**，因為行為變動為同類行為多種替換，不需解析參數

Thetawave.ai

案例三：終端機插件錯誤修正

- **Context**：終端機指令錯誤由插件修正
- **需求**：錯誤類型多種且持續擴充，不能修改插件程式碼
- **行為變動分析**：
  - 操作：修正指令
  - 輸入參數：錯誤訊息類型（語法錯誤、參數錯誤等）
  - 不同錯誤類型對應不同處理行為
- **行為變動性**：輸入比對型行為變動性
- **結論**：適用**責任鏈模式**

---

8. 設計推理的實用技巧

技巧	說明
先定位，再分類	先在系統中定位約束（force）存在於哪個操作，再判定該force屬於哪種類型
關注行為變動性細節	行為變動的類型是設計決策的關鍵，尤其是要區分「原始型」與「輸入比對型」
使用關鍵句型	利用填空句型幫助精準描述行為變動性，促使設計思考更細膩
遵循開閉原則	在擴充行為時，不修改既有程式碼，這是設計模式套用的核心需求之一

---

9. 策略模式與責任鏈模式的行為與結構關係總結表

比較面向	策略模式（Strategy）	責任鏈模式（Chain of Responsibility）
行為觸發依據	固定策略物件，行為不因輸入參數變動	根據輸入參數判斷由哪個Handler處理行為
行為變動類型	原始型行為變動性	輸入比對型行為變動性
類別結構	一個抽象策略介面，多個具體策略子類	一個抽象Handler，串連多個具體Handler形成鏈
Client作用	持有策略物件，委派行為	將request傳入Handler鏈，由鏈中節點依序處理
擴充性重點	可替換策略物件，擴充策略行為	可新增Handler節點，擴充輸入參數對應的處理行為

Thetawave.ai

10. 設計判斷關鍵句型示範

模式	句型範例	解析說明
策略模式	「在 X 操作中，存在多種行為之一，可抽換多種行為之一，如 A、B、C，且未來會持續擴充。」	簡單多行行為變動，行為切換不依賴參數
責任鏈模式	「在 X 操作執行時，必須先解析輸入參數 Y，每種參數類型對應不同處理，如 A、B、C，且未來會持續擴充。」	行為動態依參數判定，需有串列傳遞行為處理

---

11. 專業術語說明

> **物件導向分析（Object-Oriented Analysis, OOA）：**
> 針對問題領域進行分析，抽象出物件、類別、操作與關係，建立領域模型。

> **擴充性（Extensibility）：**
> 系統可在不修改既有程式碼的前提下，新增功能或行為。

> **開閉原則（Open-Closed Principle, OCP）：**
> 軟體實體（類別、模組、函式等）應該對擴充開放，對修改封閉。

策略模式與責任鏈模式雖然都屬於行為型設計模式，核心在於支援行為的擴充，但策略模式著重於在同一操作下多種行為的抽換（原始型行為變動），而責任鏈模式則專注於根據輸入參數的類型動態決定由哪個處理者執行行為（輸入比對型行為變動）。從結構與動態行為的不同，可以推導出模式適用的情境與forces，進而精準決策何時套用哪個模式，避免流於形式的誤用，提升設計品質與靈活性。

4.2 練武 - 精準區分策略＆責任鏈 Forces.mp4

🎯 物件導向設計中策略模式與責任鏈模式的區別與應用

> **策略模式**（Strategy Pattern）與**責任鏈模式**（Chain of Responsibility Pattern）是兩種常見的行為型設計模式，主要用於解決物件行為變動性的問題，但兩者適用的變動類型及設計思路有所差異。

物件導向分析與設計流程（OAD）

整體分析設計流程包含：

Thetawave.ai

1. **物件導向分析（OOA）**
   - 針對需求文本萃取類別（Class）與物件（Object），確定其屬性（Attributes）與行為（Responsibilities）。
   - 例如：交友APP中「聊天對象」為類別，行為有「開話題」。
2. **察覺與分類 Forces（設計驅動因素）**
   - **Forces** 是影響系統設計的因素，特別是行為變動性與系統擴充性需求。
   - 主要察覺三種forces：
     - **擴充性 force**（Extensibility Force）
     - **行為變動性 force（Behavioral Variation，簡稱BV）**
       - **原始型**（Primitive Behavioral Variation）
       - **輸入比對型**（Input Matching Behavioral Variation）
3. **定義問題（Problem）**
   - 由兩種或多種forces產生的衝突構成問題，例如擴充性與行為變動性間的互斥。
4. **套用設計模式**
   - 根據問題與context，選擇適當的設計模式進行重構。
5. **產生新版設計藍圖（Resulting Context）**

🛠️ 設計驅動因素（Forces）詳細說明

種類	定義與特徵	關鍵句型示例（敘述擴充或變動範圍）
擴充性 force	系統需能擴充新功能，但不修改既有程式碼。常用 **OCP 原則（開放封閉原則）** 表達。	Open for new _____，Close for existing _____。
行為變動性 BV	系統內行為會變動，需封裝變動部分。分為兩種：	
- 原始型 BV	行為變動不依賴輸入參數，直接替換不同行為實作。	在執行操作時可抽換多種行為之一，且未依賴輸入資料決定。
- 輸入比對型 BV	行為變動依賴解析輸入參數，根據輸入不同選擇不同行為。	執行操作時先解析輸入，根據輸入內容決定採用何種行為策略。

策略模式與責任鏈模式的核心差異

特色	策略模式（Strategy）	責任鏈模式（Chain of Responsibility）
適用行為變動類型	原始型行為變動	輸入比對型行為變動

行為封裝方式	將多種演算法或行為封裝成獨立類別，並透過介面統一調用	將多個處理節點串成鏈，依序判斷並處理特定輸入
動態行為決定	透過策略物件直接切換行為實作	根據輸入資料由責任鏈中不同節點決定是否處理
擴充性	可新增策略類別，替換行為實作	可新增鏈節點，擴充處理流程
設計重點	封裝行為差異，讓行為可替換	封裝多種處理節點，根據條件動態委派

交友大師APP案例分析

物件導向分析

- **類別與行為**
  - 類別：聊天對象（屬性：姓名、生日、自我介紹、年齡）
  - 行為：開話題（操作，傳入聊天對象）
- **行為變動性**
  - 開話題有三種策略，屬於行為變動性force（BV）
  - 擴充性force：可新增開話題策略而不修改既有程式碼，符合OCP原則

Forces察覺與分類

Force類型	內容說明	表示方式
行為變動性BV	有3種開話題策略，行為會根據輸入聊天對象資訊決定策略	標記為「輸入比對型BV」
擴充性force	擴充策略時不得修改既有程式碼	使用OCP表達 open for 新策略，close for 既有程式碼

問題定義

- 擴充性force 與 行為變動性force 相互衝突，需設計模式解決

設計模式選擇

- **責任鏈模式**
  - 適用輸入比對型行為變動性
  - 透過依賴反轉重構三步驟：
    1. 封裝行為變種（封裝3個策略類別）
    2. 萃取共同行為（抽象類別，定義開話題方法）
    3. 委派行為（交友大師APP委派開話題行為至責任鏈）

結果

- 新增策略時只需新增責任鏈節點，無需修改既有程式碼

- 行為依輸入聊天對象資訊決定所屬策略節點處理

Thetawave.ai

排序演算法套件案例分析

物件導向分析

- 類別：排序套件、使用者
- 行為：使用者呼叫排序功能，傳入數字陣列

Forces察覺與分類

Force類型	內容說明	表示方式
行為變動性BV	支援多種排序演算法，行為可替換	標記為「原始型BV」
擴充性force	新增排序演算法時不修改套件內部程式碼	OCP原則表達

設計模式選擇

- **策略模式**
  - 適用原始型行為變動性
  - 依賴反轉重構三步驟：
    1. 封裝行為變種（封裝各排序演算法類別）
    2. 萃取共同行為（策略介面，定義排序方法）
    3. 委派行為（排序套件委派排序行為至策略介面）

結果

- 使用者可自由替換排序策略，擴充不影響既有程式碼

男生臉型適合發型判斷案例分析

物件導向分析

- 類別：Common Line Interface（CLI）
- 行為：問使用者選擇題，根據選項判斷臉型，再推薦發型

Forces察覺與分類

Force類型	內容說明	表示方式
行為變動性BV	根據不同臉型（方臉、圓臉、菱形臉、長臉）推薦不同發型	標記為「輸入比對型BV」

擴充性force	擴充新臉型或發型時不修改既有程式碼	OCP原則表達

Thetawave.ai

設計模式選擇

- **責任鏈模式**
  - 封裝方臉、圓臉、菱形臉、長臉推薦發型為不同責任節點
  - 抽象類別定義「推薦發型」行為
  - 委派推薦行為由CLI轉交至責任鏈節點

結果

- 可新增臉型與推薦發型而不修改既有程式碼
- 根據輸入臉型選項動態決定推薦策略

設計模式辨識大師機器人案例分析

物件導向分析

- 類別：設計模式辨識大師（機器人）、Force、設計模式
- 行為：接收一組forces，判斷並回傳適用設計模式（0或1表示有無辨識）

Forces察覺與分類

Force類型	內容說明	表示方式
行為變動性BV	辨識行為有三種可能（可視為行為變動性）	標記為「輸入比對型BV」
擴充性force	擴充支援新設計模式時不修改既有程式碼	OCP原則表達

設計模式選擇

- **責任鏈模式**
  - 封裝不同設計模式辨識為責任鏈節點
  - 抽象類別定義辨識行為
  - 委派辨識行為至責任鏈節點
  - 無法辨識時回傳失敗訊息

依賴反轉之重構三步驟（適用策略模式與責任鏈模式）

Thetawave.ai

步驟編號	步驟名稱	說明
1	封裝行為變種	將不同的行為變異封裝成獨立類別（策略或責任節點）
2	萃取共同行為	抽象出共用的行為介面或抽象類別，明確定義行為規範
3	委派行為	將原本類別的行為委派給封裝的行為類別（策略或責任鏈）

設計關鍵句型示例

Force類型	關鍵句型範例（核心語意）
擴充性 force	Open for new _____, Close for existing _____
原始型行為變動性	在操作執行時，可替換多種行為，且不依賴輸入資料
輸入比對型行為變動性	執行操作前解析輸入，根據解析結果決定使用何種行為策略

重要概念強調

- **擴充性與行為變動性為設計的兩大驅動力，需妥善察覺並分類。**
- **策略模式適用原始型行為變動，強調直接替換策略物件。**
- **責任鏈模式適用輸入比對型行為變動，強調動態鏈結多個處理節點。**
- **依賴反轉三步驟為設計模式套用的標準流程，確保設計清晰且易擴充。**
- **OCP（開放封閉原則）是擴充性force的經典表達，寫出清晰的open/close範圍有助於設計判斷。**

交互流程示意（以責任鏈模式為例）

```
[使用者操作] → [Context物件執行行為] → [責任鏈節點1判斷是否處理] →
        ↳ 是 → 執行處理 → 結束
        ↳ 否 → 傳給下一節點 → 依序判斷執行
```

行為變動性與擴充性的forces在設計模式的應用中相互制約，通過策略模式或責任鏈模式的重構，能夠達成既能因應多變的行為需求，又能保持系統的開放封閉性，使軟體系統具備良好的

可維護性與可擴展性。

Thetawave.ai

🔍 行為變動辨識與設計模式套用

> **行為變動（Behavioral Variation）**指系統中某些行為在不同情境下會有不同實作的變化，需透過設計模式來封裝與管理這些變動。

行為變動類型與對應設計模式

行為變動類型	說明	對應設計模式	特點與註解
輸入比對型	根據輸入的特定類型（Forces）決定使用哪個行為變體	**責任鏈模式**（Chain of Responsibility）	依序比對輸入，找到合適的處理者；方便未來擴充新的行為。
原始型	行為變動由外部使用者決定，不依賴輸入參數	**策略模式**（Strategy Pattern）	行為封裝成多個策略類別，使用者可選擇與替換策略，符合開放封閉原則。

🧩 責任鏈模式套用流程（輸入比對型行為變動）

1. **封裝行為變動處理者（辨識者）**
   - 將所有比對行為封裝成子類別，繼承抽象辨識者類別，實作具體比對行為。
   - 例如：針對不同的 Forces 類型，分別建立對應的辨識子類。
2. **萃取共同行為**
   - 建立一個抽象類別（設計模式辨識者），定義抽象操作（辨識行為）。
   - 所有子類別繼承此抽象類別，實作具體辨識邏輯。
3. **建立責任鏈關聯**
   - 將辨識者節點串成鏈條（0 到 1 的遞迴關聯），依序傳遞輸入直到找到合適的處理者。
   - 0 到 1 的鏈條不可省略，確保鏈中每個節點都可正確呼叫下一個節點。
4. **委派行為**
   - 主要物件將辨識行為委派給責任鏈的起點，傳入輸入參數（Forces），取得辨識結果。
   - 行為本身不在主物件中實作，主物件專注委派與整合。

責任鏈模式關鍵說明

> **為什麼使用責任鏈模式？**
> 因為行為變動是「輸入比對型」，需解析輸入的 Forces 才能決定對應的設計模式，透過責任鏈模式能有效串接多個比對邏輯且方便擴充。

Thetawave.ai

🎯 策略模式套用流程（原始型行為變動）

問題背景

- **行為變動**：人工智慧決策行為有兩種變體
  1. 根據玩家狀態做決策
  2. 隨機做決策（不依賴玩家狀態）
- **行為變動非輸入比對型**，由外部使用者決定策略，不透過輸入解析決定。

對應解決方案：策略模式

1. **封裝行為變種**
   - 將不同決策行為封裝成獨立策略類別。
   - 例：PlayerStateDecision（根據玩家狀態決策）、RandomDecision（隨機決策）。
2. **萃取共同行為介面**
   - 定義策略介面（例如：DecisionStrategy），包含 makeDecision(player) 抽象方法。
   - 兩種策略類別皆實作此介面，確保接口一致。
3. **委派行為**
   - 人工智慧模組本身不實作決策邏輯，改由持有策略物件並委派呼叫。
   - 人工智慧接收玩家資料並呼叫策略物件的決策方法。

策略模式關鍵細節

項目	說明
策略介面方法	makeDecision(player)，參數為玩家資料，決策依據。
參數適用性	雖然隨機策略不使用玩家參數，但為統一接口，仍需接受該參數。
擴充性	新增決策策略時，只需新增策略類別，符合 **開放封閉原則**（Open-Closed Principle, OCP）。

🗂️ 行為變動封裝與重構三步驟（依賴反轉原則）

Thetawave.ai

步驟	內容說明
1. 封裝行為變動處	把不同的行為變動封裝成獨立類別或類型（策略、責任鏈、樣板方法等）。
2. 萃取共同行為	從這些行為變動中抽取共用的抽象介面或類別，定義統一的行為接口。
3. 委派行為	原本的物件不直接實作變動行為，改委派給封裝好的行為物件，降低耦合度。

🎮 人工智慧模組案例分析

元素	說明
物件	遊戲（Game）、人工智慧模組（AI Module）、玩家（Player）
行為	遊戲開始後，人工智慧模組啟動並做決策（makeDecision）
行為變動類型	原始型（非輸入比對型）行為變動，由使用者決定採用哪種決策策略
行為變動封裝	策略模式封裝不同決策實作，介面統一讓人工智慧模組委派呼叫
擴充性需求	新增決策行為時無需修改既有程式碼，直接新增策略類別即可，符合開放封閉原則（OCP）

重要概念重點整理

- **Forces**：影響行為變動的因素，是辨識行為變動類型的關鍵輸入。
- **Behavioral Variation（行為變動）**：系統中會變動的行為部分，需封裝管理。
- **責任鏈模式**：適用於根據輸入比對來決定行為的場景，透過鏈條串接多個處理者。
- **策略模式**：適用於行為變動由外部決定且行為間互不影響的場景，將行為封裝成策略類別。
- **依賴反轉原則**：封裝行為變動，抽象出接口，讓高層模組依賴抽象而非具體實現。
- **開放封閉原則（OCP）**：軟體對擴充開放，對修改封閉，設計模式有助於實現此原則。

行為變動類型判定流程

判斷問題	判斷結果	後續設計模式選擇
行為變動是否依賴輸入比對？	是	責任鏈模式
行為變動是否由外部使用者決定？	是（非依賴輸入）	策略模式

其他行為變動類型	未涵蓋（本課程只學兩種行為變動）	降級成最簡原始型，使用策略模式簡易版本

物件導向分析與設計實務心得

- 行為變動封裝要「一步步慢慢做」，避免遺漏，尤其是定位行為變動時需檢查操作是否完整。
- 實務中常需要反覆檢視「Context、Forces、Problem」，用以確認行為變動的本質與適合的設計模式。
- 開發過程中發現「漏抓操作」或「行為變動類型判斷錯誤」時，應立即補齊，保持思路清晰。
- 熟練設計模式辨識與套用，能使需求分析與設計過程流暢自然，減少過度思考與混亂。
- 持續練習不同題目與場景，培養空杯心態，避免過度依賴舊經驗，提升設計敏銳度與靈活度。

行為變動分析的核心關鍵句型示範

> **輸入比對型（責任鏈）示範**
> 「行為變動是輸入比對型，輸入為 Forces，透過責任鏈逐一比對，決定對應設計模式。」

> **原始型（策略）示範**
> 「行為變動是原始型，由使用者決定採用哪種行為策略，策略間互不依賴，適合策略模式封裝。」

行為變動辨識與設計模式之間的關係錯綜複雜，但通過系統性分析 Context（上下文）、Forces（影響力量）與 Problem（問題定義），能精準選擇適合的設計模式。責任鏈模式與策略模式皆透過封裝變動行為、萃取共同行為接口，再委派給具體行為物件，達成靈活擴充與低耦合設計目標。

2.7 補帖 - 無招勝有招_依賴反轉之重構三步驟.mp4

🎯 依賴反轉之重構三步驟核心解讀

依賴反轉 (Dependency Inversion) 設計原則

Thetawave.ai

> **定義：**
> 核心（高層次）模組不能依賴衍生（低層次）模組，兩者應該依賴相同的抽象。抽象不應依賴細節，細節應該依賴抽象。

- **核心模組（高層次）**：系統中重要且穩定的部分，如遊戲中的角色 (hero) 類別。
- **衍生模組（低層次）**：可變動、易擴充的部分，如角色的多種攻擊行為。
- **抽象（介面）**：介於核心與衍生模組間的橋樑，定義共同的行為規範。

問題背景：核心模組依賴衍生模組的問題

依賴方向	影響說明
核心 → 衍生	擴充衍生行為必須修改核心模組，導致核心模組維護困難、程式碼臃腫且錯誤率高

- **例子：**
  遊戲中角色 hero 直接實作攻擊行為，新增攻擊技能必須修改 hero 類別，破壞維護性。

依賴反轉之重構三步驟（Reconstruction Three Steps）

步驟	內容說明	目的與效果
1. 封裝變動處	為每個行為變種開一個類別，封裝具體行為。	把變動的行為抽離核心模組，避免核心模組直接依賴具體行為。
2. 萃取共同行為	從變種類別中抽取共同行為，定義抽象介面（放在核心模組）。	將依賴反轉，核心與衍生模組都依賴抽象介面，確保擴充性與穩定性。
3. 委派	核心模組改為依賴抽象介面，將行為委派給實作該介面的衍生模組。	核心模組不再依賴衍生模組，實現依賴反轉設計原則。

詳細解說三步驟

1. 封裝變動處（Encapsulate What Varies）

- **說明：**
  將多種不同的行為變種獨立封裝為不同類別，例如水球攻擊、火球攻擊、地球攻擊各自成為獨立類別。
- **操作簽章（簽名）設計：**
  為每個行為定義最小且恰當的操作簽章（方法參數），保持封裝隱私與精準度。
  - 水球攻擊：攻擊單一怪物 → 傳入 1 個怪物
  - 火球攻擊：攻擊 1~5 個怪物 → 傳入 1~5 個怪物

Thetawave.ai

    - 地球攻擊：隨機攻擊多個怪物 → 傳入多個怪物

- **比喻：**
  就像包裹上的標籤，操作簽章是包裹的標籤，讓外界知道包裹內裝了什麼，封裝更完整。

2. 萃取共同行為（Abstract Common Behaviors）

- **說明：**
  從上述多個類別的操作簽章中，找出共同交集，定義一個抽象介面（如 AttackType）。
    - 由於操作參數差異（1 個、1~5 個、多個怪物），取「最大需求」作交集，即使用可接受多個怪物的參數。
- **抽象介面擺放位置：**
  須放置於核心模組中，確保衍生模組依賴核心模組的介面。
- **效果：**
  抽象介面成為雙方依賴的共同基礎，實作此介面的衍生類別必須符合該抽象。

3. 委派（Delegate）

- **說明：**
  核心模組 (hero) 不再直接實作攻擊行為，而是持有該抽象介面的參考，呼叫介面方法完成攻擊。
    - hero 類別與抽象介面形成依賴或關聯關係。
    - 實際行為由衍生模組中具體類別實作。
- **結果：**
  核心模組與衍生模組的依賴方向由「核心→衍生」反轉成「衍生→核心抽象」，達成依賴反轉設計原則。

依賴反轉與設計模式的關係

設計模式	重點	依賴反轉三步驟的角色
策略模式	行為變動與擴充	透過三步驟完成抽象介面，讓核心角色依賴介面，行為實作獨立。
責任鏈模式	多行為變種串聯組合	依賴反轉三步驟完成抽象分離，並額外建立遞迴關聯串起多個處理者。

- **觀察：**
  設計模式中解決變動性與擴充性的問題，核心方法皆是利用依賴反轉三步驟完成抽象分離與依賴反轉。

重要概念與術語整理

Thetawave.ai

概念	解釋
Context（上下文）	進行重構或應用設計模式的範圍，如 hero 類別中攻擊行為的部分。
變種（Variant）	同類型行為中最小變化單位，如不同攻擊技能或角色。
行為變動性（Behavioral Variability）	行為可變動的特性，需封裝與抽象以因應擴充。
擴充性（Extensibility）	可新增行為變種而不修改既有核心程式碼的能力。
操作簽章（Method Signature）	方法的名稱與參數列表，代表行為的接口規範。
抽象介面（Interface/抽象類別）	定義共同行為的契約，放置於核心模組，供核心與衍生模組依賴。
委派（Delegation）	核心模組將行為執行權交給抽象介面實作，由衍生模組負責具體行為。

操作簽章取交集的原則

- 取交集時要「聽最多需求的那一方」，確保新介面能涵蓋所有衍生類別的行為需求。
- 例如：
  水球攻擊只需 1 個怪物參數，火球攻擊 1~5 個怪物，地球攻擊多個怪物，介面簽章定為「多個怪物」可覆蓋所有需求。

依賴反轉之重構三步驟執行流程

流程步驟	執行內容
定位 Context	確認待分離的系統範圍與變動點，如 hero 類別的攻擊行為。
封裝每個行為變種	為每個具體行為開新類別、定義操作簽章並實作行為。
萃取共同行為抽象介面	根據操作簽章交集設計抽象介面，介面放入核心模組。
介面實作	由封裝的變種類別實作抽象介面。
核心模組委派行為	hero 類別持有抽象介面參考，呼叫介面方法完成行為。
依賴方向反轉	依賴關係由「核心→衍生」變成「衍生→核心抽象」，實現依賴反轉原則。

依賴反轉三步驟的應用提醒

建議事項	理由
精準定位 Context	不可亂套模式，確保重構範圍明確，避免設計混亂。

操作簽章定義要小氣且精準	小而精的簽章有助於後續抽象精準，避免過度泛化或不必要複雜度。
初期採用保守抽象策略	先學會基礎抽象再逐步增加彈性，避免一開始就追求過度靈活導致設計失衡。
慢工出細活	設計重點在於細心與精準，而非追求快速完成，類比機車駕照直線七秒的平衡技巧。

Save to Notebook ave.ai

依賴反轉三步驟在策略與責任鏈模式中的對比

模式	抽象位置	變種類別位置	依賴方向	額外結構特點
策略模式	核心模組（介面）	衍生模組	衍生模組依賴核心抽象	無特殊連結
責任鏈模式	核心模組（介面）	衍生模組	衍生模組依賴核心抽象	變種類別間有遞迴關聯形成鏈條

- 責任鏈模式允許多個行為變種串聯參與決策，需多拉遞迴關聯來組合責任鏈。

依賴反轉設計的意涵

依賴反轉將系統中不穩定且易變的部分獨立封裝，並抽象成介面讓核心系統依賴該介面，促使系統擴充時無需改動核心邏輯，保證核心模組的穩定性與可維護性。所有衍生模組皆依賴核心模組中的抽象介面，這種雙向依賴抽象的結構使得系統既靈活又具備良好架構，並且可以在不破壞核心的情況下自由擴展衍生行為。